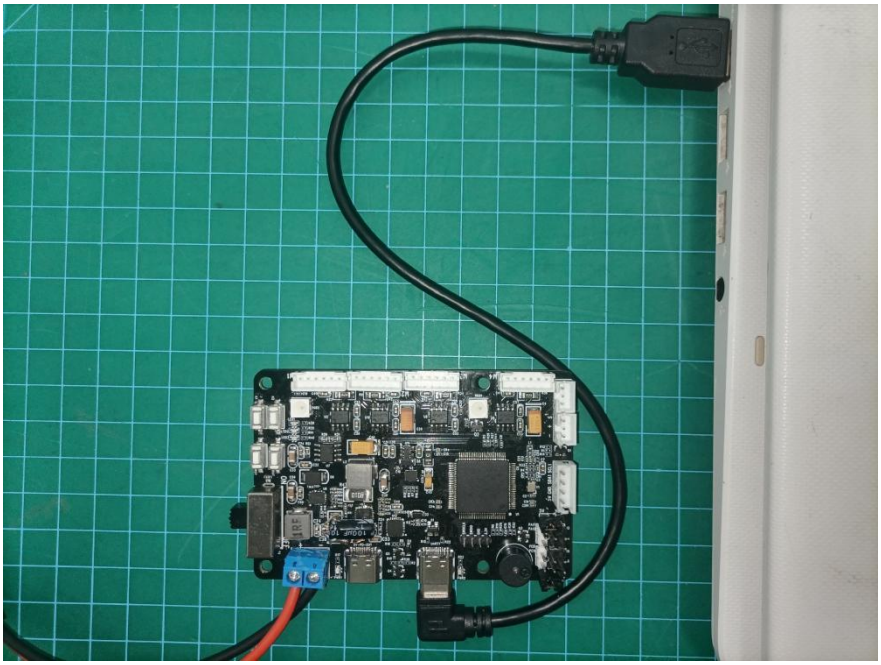
Lesson 15 RRCLite-Host Computer Communication Source Code Analysis

1. Program Logic

This section mainly analyzes the source code for receiving and processing communication between the RRCLite development board and the host computer.

2. Hardware Connection

The hardware connection diagram is as below:



3. Program Download

The example for this section is “RosRobotControllerLite_ros”.

4. Program Outcome

After porting the ROS package, you can control the car's LED lights on and off, activate the buzzer, rotate the encoder motors, rotate PWM servos, rotate bus

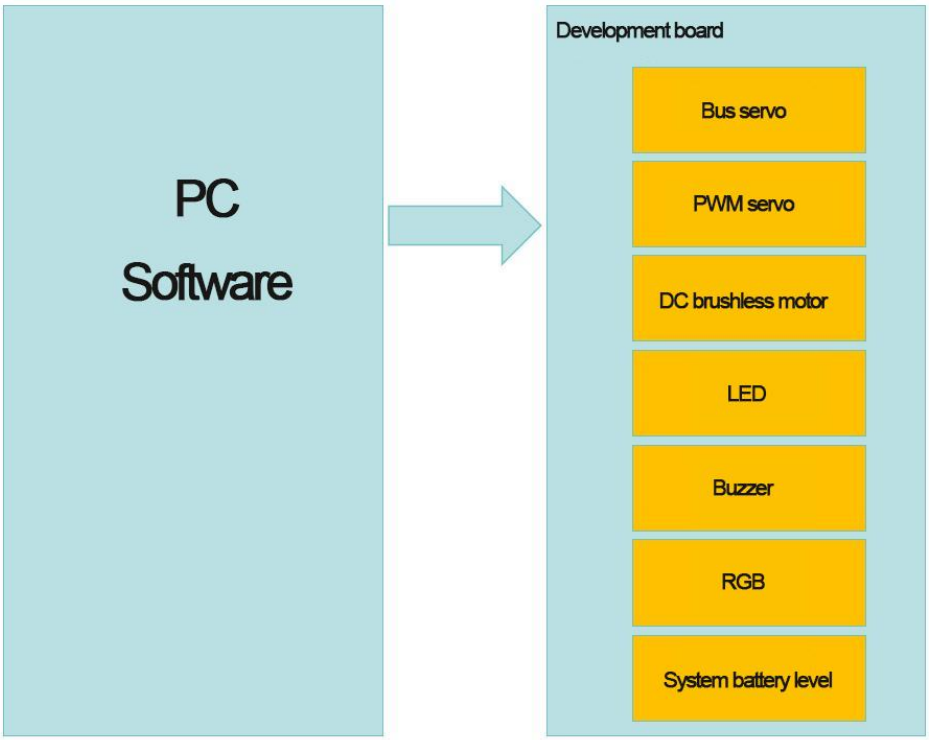
servos, match RGB light color, and check system power by calling the host computer interface.

5. PIN Info Configuration

The development board's microcontroller uses UART1 serial port, which communicates with the host computer via a conversion circuit to USB signal. In STM32CubeMX software, you can view pin information. According to the diagram below, the corresponding chip pins are: PA9 and PA10.

6. Source Code Analysis

The program source code for this session is located in “RosRobotControllerLite_ros”. The program framework is designed according to the functionality, as shown in the diagram below.



The messages sent by the host computer to control the development board are divided into seven categories: LED control messages, buzzer control messages, encoder motor control messages, PWM servo control messages, bus servo control messages, RGB light control messages, and power alarm threshold modification control messages.

6.1 Reception of LED Control Messages

The packet_led_handle() function in the Hiwonder/System/packet_handle.c file is the callback function for handling LED serial commands. Upon receiving LED control messages, it enters this callback function to execute LED control. The specific program is shown in the diagram below:

```
void packet_led_handle(struct PacketRawFrame *frame)
{
    LedCommandTypeDef *cmd = (LedCommandTypeDef*)frame->data_and_checksum;
    uint8_t led_id = cmd->led_id - 1;
    if(led_id < 1)
        led_flash(leds[led_id], cmd->on_time, cmd->off_time, cmd->repeat);
    }
}
```

The parameter frame of packet_led_handle() is a pointer parameter used to store the control information sent by the host computer. This data is strictly stored according to the LED control structure LedCommandTypeDef. Therefore, the data stored in the frame can be directly converted into the LED control structure to obtain the correct control parameters. The program flow is as follows:

① Convert the data into the LED control structure.

② Check if the LED ID number is correct:

If correct, use the structure data to control the LED change.

If incorrect, skip directly.

The definition of the LED control structure LedCommandTypeDef is as follows:

```
typedef struct {
    uint8_t led_id;
    uint16_t on_time;
    uint16_t off_time;
    uint16_t repeat;
} LedCommandTypeDef;
```

For specific parameter details, please refer to the LED control section in the 'Lesson 13 RRCLite Communication Protocol with the Host Computer Analysis' document.

6.2 Reception of Buzzer Control Messages

The packet_buzzer_handle() function in the **Hiwonder/System/packet_handle.c** file is the callback function for handling buzzer serial commands. Upon receiving a buzzer control message, it enters this callback function to execute buzzer control. The specific program is shown in the diagram below:

```
void packet_buzzer_handle(struct PacketRawFrame *frame)
{
    BuzzerCommandTypeDef *cmd = (BuzzerCommandTypeDef*)frame->data_and_checksum;
    buzzer_didi(buzzers[0], cmd->freq, cmd->on_time, cmd->off_time, cmd->repeat);
}
```

The parameter 'frame' is strictly stored according to the BuzzerCommandTypeDef buzzer control structure, so converting the data stored in 'frame' directly into the buzzer control structure retrieves the correct control parameters. The program flow is as follows:

①：Convert the data into the buzzer control structure.

②：Use the structure data to control the buzzer activation.

The definition of the BuzzerCommandTypeDef buzzer control structure is as follows:

```
typedef struct {
    uint16_t freq;
    uint16_t on_time;
    uint16_t off_time;
    uint16_t repeat;
} BuzzerCommandTypeDef;
```

For specific parameter details, please refer to the buzzer control section in the 'Lesson 13 RRCLite Communication Protocol with the Host Computer Analysis' document.

6.3 Reception of Encoder Motor Control Messages

The packet_motor_handle() function in the Hiwonder/System/packet_handle.c file is the callback function for handling encoder motor serial commands. Upon receiving an encoder motor control message, it enters this callback function to execute encoder motor control. The specific program is shown in the diagram below:

```
static void packet_motor_handle(struct PacketRawFrame *frame)
{

    switch(frame->data_and_checksum[0]) {
        case 0: {
            MotorSingalCtrlCommandTypeDef *mscc = (MotorSingalCtrlCommandTypeDef *)frame->data_and_checksum;
            motors[mscc->motor_id]->pid_controller.set_point = mscc->speed;
            break;
        }
        case 1: {
            MotorMutilCtrlCommandTypeDef *mmcc = NULL;
            mmcc = (MotorMutilCtrlCommandTypeDef *)frame->data_and_checksum;
            for(int i = 0; i < mmcc->motor_num; ++i) {
                motors[mmcc->element[i].motor_id]->pid_controller.set_point = mmcc->element[i].speed;
            }
            break;
        }
        case 2:  {
            MotorSingalStopCommandTypeDef *mssc = (MotorSingalStopCommandTypeDef *)frame->data_and_checksum;
            motors[mssc->motor_id]->pid_controller.set_point = 0;
            break;
        }
        case 3: {
            MotorMultiStopCommandTypeDef *mmsc = (MotorMultiStopCommandTypeDef *)frame->data_and_checksum;
            for(int i = 0; i < 4; ++i) {
                if(mmsc->motor_mask & (0x01 << i)) {
                    motors[i]->pid_controller.set_point = 0;
                }
            }
            break;
        }
```

```
    case 5: { //Motor type switching
        MotorTypeCtlTypeDef *mmsc = (MotorTypeCtlTypeDef *)frame->data_and_checksum;
        MotorTypeEnum type = MOTOR_TYPE_JGB520;
            // printf("type:%d",mmsc->type);
        if(mmsc->type == MOTOR_TYPE_JGB520){
            type = MOTOR_TYPE_JGB520;
        }else if(mmsc->type == MOTOR_TYPE_JGB37){
            type = MOTOR_TYPE_JGB37;
        }else if(mmsc->type == MOTOR_TYPE_JGA27){
            type = MOTOR_TYPE_JGA27;
        }else if(mmsc->type == MOTOR_TYPE_JGB528){
            type = MOTOR_TYPE_JGB528;
        }else {
            type = MOTOR_TYPE_JGB520;
        }
        for(int i = 0 ; i < 4 ; i++){
            set_motor_type(motors[i], type);
        }
        break;
    }
    default:
        break;
}
```

To streamline the control of encoder motors, the first control data represents the sub-command for motor control. There are a total of 5 sub-commands, each corresponding to a specific structure and functionality as follows:
(MotorSingalCtrlCommandTypeDef) Single motor movement,
(MotorMutilCtrlCommandTypeDef) Multiple motor movement,
(MotorSingalStopCommandTypeDef) Single motor stop,
(MotorMultiStopCommandTypeDef) Multiple motor stop,
(MotorTypeCtlTypeDef) Motor type switching function.
The parameter 'frame' is strictly stored according to the corresponding structure of encoder motor control based on each sub-command. Therefore, converting the data stored in 'frame' directly into the corresponding encoder motor control structure retrieves the correct control parameters. The program flow is as follows:

①：Read the first data to determine which sub-function it corresponds to.
②：Enter the specified sub-function section and convert the incoming data into the corresponding sub-function structure parameters.
③：Use the structure parameters to control the movement of the encoder motor.

The definitions of various structures for encoder motor control are as follows:

```
//Multiple motor controls
typedef struct {
    uint8_t cmd;
    uint8_t motor_num;
    struct {
        uint8_t motor_id;
        float speed;
    } element[];
} MotorMutilCtrlCommandTypeDef;

//Single motor control
typedef struct {
    uint8_t cmd;
    uint8_t motor_id;
    float speed;
} MotorSingalCtrlCommandTypeDef;

//Single motor stops
typedef struct {
    uint8_t cmd;
    uint8_t motor_id;
} MotorSingalStopCommandTypeDef;
```

```
//Motor type switching
typedef struct {
    uint8_t func;
    uint8_t type;
} MotorTypeCtlTypeDef;
```

For specific parameter details, please refer to the encoder motor control section in the 'Lesson 13 RRCLite Communication Protocol with the Host Computer Analysis' document.

6.4 Reception of PWM Servo Control Messages

The packet_serial_servo_handle() function in the Hiwonder/System/packet_handle.c file serves as the callback function for handling PWM servo serial commands. Upon receiving a PWM servo control message, the program enters this callback function to execute PWM servo control. The specific program is illustrated in the diagram below:

```
/**
* @brief PWM servo serial command callback processing
* @param Frame data frame
* @retval void
*/
static void packet_pwm_servo_handle(struct PacketRawFrame *frame)
{
    switch(frame->data_and_checksum[0]) {
        case 0x01: {    //Multiple servo control
        case 0x03: {    //Single servo control
        case 0x05: { // Read the current position of the servo motor
        case 0x07: { // Set servo deviation
        case 0x09: { // Read servo deviation
        default:
            break;
    }
}
```

Firstly, the program extracts the sub-command parameters stored in the parameter header element, and then enters the corresponding sub-code snippet after comparison.

The parameter 'frame' is strictly stored according to the control structure corresponding to the PWM servo sub-function. By directly converting the data stored in 'frame' into the PWM servo control structure, the correct control parameters can be obtained. The program flow is as follows:

1. Read the first data to determine which sub-function it corresponds to.
2. Enter the specified sub-function section and convert the incoming data into the corresponding sub-function structure parameters.
3. Use the structure parameters to control the movement of the PWM servo.

The definition for each structure of PWM servo control is as follows:

```
/* PWM servo motor */
typedef struct {
    uint8_t cmd;
    uint8_t servo_id;
    uint8_t args[];
} PWM_ServoCommandTypeDef;

typedef struct {
    uint8_t cmd;
    uint16_t duration;
    uint8_t servo_id;
    uint16_t pulse;
} PWM_ServoSetPositionCommandTypeDef;

typedef struct {
    uint8_t cmd;
    uint16_t duration;
    uint8_t servo_num;
    struct {
        uint8_t servo_id;
        uint16_t pulse;
    } elements[];
} PWMServoSetMultiPositionCommandTypeDef;
```

6.5 Reception of Power Alarm Threshold Modification Control Messages

The packet_battery_limit_handle( ) function in the Hiwonder/System/packet_handle.c file serves as the callback function for handling bus servo serial commands. Upon receiving a bus servo control message, the program enters this callback function to execute bus servo

control. The specific program is illustrated in the diagram below:

```
/**
* @brief Battery alarm setting callback processing
* @param Frame data frame
* @retval void
*/
static void packet_battery_limit_handle(struct PacketRawFrame *frame)
{
    BatteryWarnTypeDef *cmd = (BatteryWarnTypeDef*)frame->data_and_checksum;
    switch(frame->data_and_checksum[0]) {
        case 1: {
            change_battery_limit(cmd->limit);
        }break;
        default:
            break;
    }
}
```

The processing logic of this function is highly similar to the previous one.

6.6 Reception of RGB Light Control Messages

The packet_RGB_Ctl_handle( ) function in the Hiwonder/System/packet_handle.c file serves as the callback function for handling RGB light serial commands. Upon receiving a bus servo control message, the program enters this callback function to execute RGB light control. The specific program is illustrated in the diagram below:

```
/**
* @brief RGB control
* @param Frame data frame
* @retval void
*/
static void packet_RGB_Ctl_handle(struct PacketRawFrame *frame)
{
    RGBCtlTypeDef *cmd = (RGBCtlTypeDef*)frame->data_and_checksum;
    switch(cmd->id) {
        case 0: {
//            for(int i = 0 ; i < Pixel_S1_NUM ; i++)
//            {
//                set_id_rgb_color(i , &cmd->data[i*3]);
//            }
            set_rgb_color(cmd->data);
        }break;

        case 1:
        case 2: {
            set_id_rgb_color((cmd->id-1) , cmd->data);
        }break;

        default:
            break;
    }
}
```

In the program, function codes 1 and 2 have the same effect. You can perform secondary development based on your own needs.

Additionally, it is worth noting that unlike LED lights, the IDs of RGB lights start from 0.

6.7 Reception of Bus Servo Control Messages

The packet_serial_servo_handle() function in the Hiwonder/System/packet_handle.c file serves as the callback function for handling bus servo serial commands. Upon receiving a bus servo control message, the program enters this callback function to execute bus servo control. The specific program is illustrated in the diagram below:

```
static void packet_serial_servo_handle(struct PacketRawFrame *frame)
{
    PacketReportSerialServoTypeDef report;
    switch(frame->data_and_checksum[0]) {
        case 0x01: { /* Steering gear control */
            SerialServoSetPositionCommandTypeDef *cmd = (SerialServoSetPositionCommandTypeDef *)frame->data_and_checksum;
            for(int i = 0; i < cmd->servo_num; i++) {
                serial_servo_set_position(&serial_servo_controller, cmd->elements[i].servo_id, cmd->elements[i].position, cmd->duration);
            }
            break;
        }
        case 0x03: { /* Stop the servo motor */
            SerialServoMultiCommandTypeDef *cmd = (SerialServoMultiCommandTypeDef *)frame->data_and_checksum;
      for(int i = 0; i < cmd->servo_num; i++) {
        serial_servo_stop(&serial_servo_controller, cmd->args[i]);
      }
            break;
        }
        case 0x05: { /* Position reading */
            int16_t position = 0;
            SerialServoCommandTypeDef *cmd = (SerialServoCommandTypeDef *)frame->data_and_checksum;
            packet_serial_servo_report_init(&report, cmd->servo_id, cmd->cmd,  serial_servo_read_position(&serial_servo_controller, cmd->s
            memcpy(report.args, &position, 2);
            packet_transmit(&packet_controller, PACKET_FUNC_BUS_SERVO, &report, 5);
            break;
        }
```

```
        case 0x01: { /* Steering gear control */
        case 0x03: { /* Stop the servo motor */
        case 0x05: { /* Position reading */
        case 0x07: { /* Input voltage reading */
        case 0x09: { /* Temperature Reading */
        case 0x0B: { /* Unloading power */
        case 0x0C: { /* Load power */
        case 0x0D: { /* Power status reading */
        case 0x10: { /* ID writing */
        case 0x12: { /* ID reading */
        case 0x20: { /* deviation adjustment */
        case 0x22: { /* Deviation reading */
        case 0x24: { /* Deviation preservation */
        case 0x30: { /* Deviation preservation */
        case 0x32: { /* Position restriction reading */
        case 0x34: { /* Voltage limit setting */
        case 0x36: { /* Voltage limit reading */
        case 0x38: { /* Temperature limit setting */
        case 0x3A: { /* Temperature limit reading */
        default:
            break;
    }
```

Bus servos have multiple sub-commands corresponding to different sub-functions, with the first control data representing the sub-command for motor control.

The parameter 'frame' is strictly stored according to the control structure

corresponding to the bus servo sub-function. If it's a sub-command for controlling motion, it's stored according to the SerialServoSetPositionCommandTypeDef structure to store control parameters. By directly converting the data stored in 'frame' into the bus servo control structure, the correct control parameters can be obtained. The program flow is as follows:

1. Read the first data to determine which sub-function it corresponds to.
2. Enter the specified sub-function section and convert the incoming data into the corresponding sub-function structure parameters.
3. Use the structure parameters to control the movement of the bus servo.

The definition of the bus servo control structure SerialServoSetPositionCommandTypeDef is as follows:

```
/* Serial port servo motor */
typedef struct {
    uint8_t cmd;
    uint8_t servo_id;
    uint8_t args[];
} SerialServoCommandTypeDef;

/* Serial port servo motor */
typedef struct {
    uint8_t cmd;
    uint8_t servo_num;
  uint8_t args[];
} SerialServoMultiCommandTypeDef;
/* Serial port servo motor */
typedef struct {
    uint8_t cmd;
    uint16_t duration;
    uint8_t servo_num;
    struct {
        uint8_t servo_id;
        uint16_t position;
    } elements[];
} SerialServoSetPositionCommandTypeDef;
```

For specific parameter details, please refer to the bus servo control section in the 'Lesson 13 RRCLite Communication Protocol with the Host Computer Analysis' document.

6.8 Source Code Analysis of Serial Protocol Reception Task

The packet_rx_task_entry() function in the Hiwonder/Porting/packet_porting.c file serves as the entry point for the serial protocol reception task. This task

handles messages received from the serial port and executes control. The program logic is as follows:

①：Initialize and clear the semaphore in the queue.

②：Register the callback function packet_uart_error_callback() for handling serial port receive error events.

③：Register callback functions corresponding to serial port message function codes with the message controller.

④：Start the hardware reception of the serial port and register the message processing callback function.

⑤：Loop for receiving and processing：

（1）Wait to acquire the semaphore. When a message is received, a semaphore is sent in the interrupt, which can be obtained here.

（2）Process the message and control the car functions.

The reception task program is as below:

```
void packet_tx_task_entry(void *argument)
{
    for(;;) {
        osSemaphoreAcquire(packet_tx_idleHandle, osWaitForever); /* Waiting to send idle signal */
        osStatus_t status = osMessageQueueGet(packet_tx_queueHandle, &packet_controller.tx_dma_buffer, NULL, osWaitForever); /* Retrieve d
        if(osOK == status) {
            HAL_UART_RegisterCallback(&huart1, HAL_UART_TX_COMPLETE_CB_ID, packet_dma_transmit_finished); /* Registration DMA reception co
            HAL_UART_Transmit_DMA(&huart1, (uint8_t*)packet_controller.tx_dma_buffer, packet_controller.tx_dma_buffer->data_length + 5);
        }
    }
}
```

Among them, the packet_uart_error_callback( ) function prevents serial port failure due to startup issues by restarting when a serial port error occurs. The function is defined as follows:

```
static void packet_uart_error_callback(UART_HandleTypeDef *huart)
{
    HAL_UART_AbortReceive(&huart3);

    HAL_UART_RegisterRxEventCallback(   &huart3,
                                        packet_dma_receive_event_callback);

    HAL_UARTEx_ReceiveToIdle_DMA(   &huart3,
                    packet_controller.rx_dma_buffers[packet_controller.rx_dma_buffer_index],
                                    PACKET_RX_DMA_BUFFER_SIZE);
}
```

The packet_handle_init() function registers the callback functions for the corresponding function codes. The function is defined as follows:

```
void packet_handle_init(void)
{
    packet_register_callback(&packet_controller, PACKET_FUNC_LED, packet_led_handle);
    packet_register_callback(&packet_controller, PACKET_FUNC_BUZZER, packet_buzzer_handle);
    packet_register_callback(&packet_controller, PACKET_FUNC_MOTOR, packet_motor_handle);
    packet_register_callback(&packet_controller, PACKET_FUNC_BUS_SERVO, packet_serial_servo_handle);
    packet_register_callback(&packet_controller, PACKET_FUNC_PWM_SERVO, packet_pwm_servo_handle);
    packet_register_callback(&packet_controller, PACKET_FUNC_SYS, packet_battery_limit_handle);
    packet_register_callback(&packet_controller, PACKET_FUNC_RGB, packet_RGB_Ctl_handle);
#if ENABLE_OLED
  packet_register_callback(&packet_controller, PACKET_FUNC_OLED, packet_oled_handle);
#endif
}
```

The packet_start_recv() function starts the serial port reception hardware and registers the interrupt handling callback function. The function is defined as follows:

```
void packet_start_recv(void)
{
    HAL_UART_AbortReceive(&huart1);
    HAL_UART_RegisterCallback(&huart1, HAL_UART_ERROR_CB_ID, packet_uart_error_callback);
    HAL_UART_RegisterRxEventCallback(&huart1, packet_dma_receive_event_callback); /* Register to receive event callbacks */
    /* Use ReceptToIdle_DMA for reception, which interrupts when DMA cache is full or interrupts when reception is idle and triggers a receive event callback */
    HAL_UARTEx_ReceiveToIdle_DMA(&huart1, packet_controller.rx_dma_buffers[packet_controller.rx_dma_buffer_index], PACKET_RX_DMA_BUFFER_SIZE);
    /* Start receiving */
}
```

The packet_dma_receive_event_callback() function is triggered during an interrupt. It pushes the received data into the data buffer and sends a semaphore to start the processing program in the reception task. The function is defined as follows:

```
static void packet_dma_receive_event_callback(UART_HandleTypeDef *huart, uint16_t length)
{
    int cur_index = packet_controller.rx_dma_buffer_index; /* Retrieve the index number of the current DMA cache */
    packet_controller.rx_dma_buffer_index ^= 1;
    HAL_UART_AbortReceive(&huart1);
    HAL_UARTEx_ReceiveToIdle_DMA(&huart1, packet_controller.rx_dma_buffers[packet_controller.rx_dma_buffer_index], PACKET_RX_DMA_BUFFER_SIZE);
    lwrb_write(packet_controller.rx_fifo, packet_controller.rx_dma_buffers[cur_index], length); /* Write the received data into the FIFO ring */
    osSemaphoreRelease(packet_rx_not_emptyHandle); /* Set the receiving buffer to receive non empty signals */
}
```

The received data is ultimately processed in the packet_recv() function within the reception task. The packet_recv() function handles the serial protocol by analyzing the received serial data. After parsing the correct data, it calls the callback function for the corresponding function code to process the control message in the data segment. The program code is as follows:

```
void packet_recv(struct PacketController *self)
{
    uint8_t data[PACKET_PARSE_BUFFER_SIZE];
    size_t avaliable = lwrb_get_full(self->rx_fifo);
    size_t readed_len = 0;
    uint8_t crc = 0;
    while(avaliable > 0) {
        avaliable = avaliable > PACKET_PARSE_BUFFER_SIZE ? PACKET_PARSE_BUFFER_SIZE : avaliable;
        readed_len = lwrb_read(self->rx_fifo, data, avaliable);
```

```
        for(int i = 0; i < readed_len; ++i) {
            printf("%X ", data[i]);
            switch(self->state) {
                case PACKET_CONTROLLER_STATE_STARTBYTE1: /* Processing frame header markers1 */
                    self->state = PROTO_CONST_STARTBYTE1 == data[i] ? PACKET_CONTROLLER_STATE_STARTBYTE2 : PACKET_CONTROLLER_STATE_STARTBYTE1;
                    break;
                case PACKET_CONTROLLER_STATE_STARTBYTE2: /* Processing frame header markers2 */
                    self->state = PROTO_CONST_STARTBYTE2 == data[i] ? PACKET_CONTROLLER_STATE_FUNCTION : PACKET_CONTROLLER_STATE_STARTBYTE1;
                    break;
                case PACKET_CONTROLLER_STATE_FUNCTION: /* Processing frame function number */
                    self->state = PACKET_FUNC_NONE > data[i] ? PACKET_CONTROLLER_STATE_LENGTH : PACKET_CONTROLLER_STATE_STARTBYTE1;
                    if(PACKET_CONTROLLER_STATE_LENGTH == self->state) {
                        self->frame.function = data[i];
                    }
                    break;
                case PACKET_CONTROLLER_STATE_LENGTH: /* Processing frame data length */
                    self->frame.data_length = data[i];
                    self->state = (0 == self->frame.data_length) ? PACKET_CONTROLLER_STATE_CHECKSUM : PACKET_CONTROLLER_STATE_DATA;
                    self->data_index = 0;
                    break;
```

```
                case PACKET_CONTROLLER_STATE_LENGTH: /* Processing frame data length */
                    self->frame.data_length = data[i];
                    self->state = (0 == self->frame.data_length) ? PACKET_CONTROLLER_STATE_CHECKSUM : PACKET_CONTROLLER_STATE_DATA;
                    self->data_index = 0;
                    break;
                case PACKET_CONTROLLER_STATE_DATA: /* Processing frame data */
                    self->frame.data_and_checksum[self->data_index] = data[i];
                    ++self->data_index;
                    if(self->data_index >= self->frame.data_length) {
                        self->state = PACKET_CONTROLLER_STATE_CHECKSUM;
                    }
                    break;
                case PACKET_CONTROLLER_STATE_CHECKSUM: /* Processing verification values */
                    self->frame.data_and_checksum[self->frame.data_length] = data[i];
                    crc = checksum_crc8((uint8_t*)&self->frame.function, self->frame.data_length + 2);
                    printf("crc:%d\r\n",crc);
                    if(crc == self->frame.data_and_checksum[self->frame.data_length]) { /* Verification failed, skip execution */
                        if(NULL != self->handles[self->frame.function]) {
                            led_flash(leds[2] , 10 , 10 , 1);
            self->handles[self->frame.function](&self->frame);
                        }
                    }
                    memset(&self->frame, 0, sizeof(struct PacketRawFrame));
                    self->state = PACKET_CONTROLLER_STATE_STARTBYTE1;
                    break;
                default:
          self->state = PACKET_CONTROLLER_STATE_STARTBYTE1;
                    break;
            }
        }
        avaliable = lwrb_get_full(self->rx_fifo);
    }
}
```

In the checksum verification segment of the above program, the callback function for the corresponding function code is called, achieving the host computer control functionality. The code snippet is shown below:

```
            case PACKET_CONTROLLER_STATE_CHECKSUM: /* Processing verification values */
                self->frame.data_and_checksum[self->frame.data_length] = data[i];
                crc = checksum_crc8((uint8_t*)&self->frame.function, self->frame.data_length + 2);
                printf("crc:%d\r\n",crc);
                if(crc == self->frame.data_and_checksum[self->frame.data_length]) { /* Verification failed, skip execution */
                    if(self->handles[self->frame.function] != NULL) {
                      self->handles[self->frame.function](&self->frame);
                    }
                }
                memset(&self->frame, 0, sizeof(struct PacketRawFrame));
                self->state = PACKET_CONTROLLER_STATE_STARTBYTE1;
                break;
            default:
```